P2. Assignment 2.1 Pretty Good Privacy with Gnupg -Gradescope-

Pretty Good Privacy with GPG

Objectives

The use of PGP/GPG is widespread in the cybersecurity community, it is true that it is not the only method to protect email communications, but every cybersecurity person has their PGP/GPG key pair. In this lab, you will generate a key pair for yourself, and learn the basics of how to generate, sign, and manage PGP keys, as well as encrypt and sign messages.

Task 0: GPG

There are many tools that support the PGP standard. In the remainder of this document we will provide a brief tutorial for using GNU Privacy Guard (GnuPG, or simply GPG), which is a free, open-source, command line implementation of PGP. This assignment is designed to be completed using Linux. but there are other tools to use (GPG for Mac OSLinks to an external site., GPG4WinLinks to an external site.).

To install GPG under a Debian-based Linux distro (such us ubuntu), simply run:

$ sudo apt install gnupg

On recent Linux distributions, such as Ubuntu 24.04, this will install GPG version 2 , which is what this assignment will assume you are using. On older Linux distributions, such as Ubuntu 18.04, version 2 of GPG can be installed via the "gnupg2" package instead (note that this method also changes the command to invoke GPG from "gpg" to "gpg2"). Other installation options for Linux, Windows, and macOS are available on the GnuPG homepageLinks to an external site. to an external site.

You can find a GPG tutorial hereLinks to an external site., and there are so many other you can use.

Task 1: For your eyes ONLY

Create a key pair for yourself that uses RSA encryption, use 4096 bit key, and includes your name and @northeastern.edu email address.

Export your public key in ASCII armor format to a file and **get three of your classmates to sign it**, hence verifying that belongs to you. Once you have it signed, don't forget to import these signatures back into your local keyring!

You will probably also need to sign some your classmates public keys as well. This involves getting a copy of their public key, importing it into your local GPG, signing it, exporting it in ASCII Armor format, and sending the signed key back to them.

Export your public key in ASCII Armor format one last time (i.e., after it has been signed by -at least- three of your classmates) to a file named userid.key.pub. (your userid from your userid@northeastern.edu email address). You will turn in this file as one of your deliverables.

Now, you are about to create your first encrypted message using your own public key. This means that you will be the only one who can decrypt the actual message, because you are the only one who has the secret key associated to it. Name your encrypted message userid.formyeyesonly.gpg, and this will also be a deliverable of this task. When we try to decrypt the ciphertext we will obtain something like this:

$ gpg -d message.txt.gpg

gpg: encrypted with 256-bit ECDH key, ID EC0493472C23A5AF, created 2024-06-13

"CY5001 Class Key <cy5001@none.com>"

gpg: decryption failed: No secret key

However, you should be able to decrypt it using the same command. Verify that the message is decrypted on your side.

WARNING: Back Up Your GPG Private Key You will need the GPG key pair you generate during this project during the semester. Thus, it is critical that you not lose your keys! You should make a backup of your private key, using the gpg --armor --export-secret-keys command, and store the backup in a secure, reliable location.

Task 2: CY5001 Class Public key

Now, consider the public key of the CY5001 Class (1D9E018225193CC3):

$ gpg --export --armor 1D9E018225193CC3

-----BEGIN PGP PUBLIC KEY BLOCK-----

mDMEZmtJ2RYJKwYBBAHaRw8BAQdA/+bwmSrYDcExWJmzN2d0puTqQLBZS/G+ksLk

2MtFT9G0IkNZNTAwMSBDbGFzcyBLZXkgPGN5NTAwMUBub25lLmNvbT6IlgQTFgoA

PhYhBKaMsKrIws1tHvBJdB2eAYIlGTzDBQJma0nZAhsDBQkDwmcABQsJCAcDBRUK

CQgLBRYCAwEAAh4BAheAAAoJEB2eAYIlGTzDlmEBAORsasRwLIZzJbY+xxocN/Lb

YbpdQor4PJREikqpO/RpAP40lIiG+qvpv0frI7I47SfjgHxhY5YEhwSbL7giIq16

Arg4BGZrSdkSCisGAQQBl1UBBQEBB0AR7qWx5/ahDY3GT3bCNKYi/7nsUCaBErF1

0tGq0R36RgMBCAeIeAQYFgoAIBYhBKaMsKrIws1tHvBJdB2eAYIlGTzDBQJma0nZ

AhsMAAoJEB2eAYIlGTzDZp4A/0kxMjmafdUjfPm/SvpsU6+YgIMJg4db2vTzsKbb

YzngAQDPlNuQA9jXZXjSG8sURepp3SutdnHuuYIkSBGu54P/BQ==

=uayt

-----END PGP PUBLIC KEY BLOCK-----

You can import the key Copy and Pasting the previous armor version, or importing from the keyserver.

```
$ gpg --keyserver keyserver.ubuntu.com --recv-key 1D9E018225193CC3

gpg: key 1D9E018225193CC3: "CY5001 Class Key <cy5001@none.com>"

gpg: Total number processed: 1

gpg:           unchanged: 1
```

Create a file named userid.confidential.gpg that is encrypted with the CY5001 class public key. Your message to send will be based on a txt file, which you create, and contains (1) your first and last name, (2) Your key type and KeyID, and (3) the string "CY5001 ". For example, a valid message.txt might look like this:

```
Marisa Sonrisa

rsa4096/C9A56A7380FF3C77

CY5001 <Fall | Spring | Summer> <year>
```

The ciphertext you just created is confidential and only us (the owners of the CY5001 class secret key) will be able to decrypt it. If you try to decrypt the message yourself, you should confirm that you do not have the secret key to decrypt it (although you know the content).

Now, we are going a step further, creating an authenticated message, which the receiver can also verify that it comes from you and from you only. GnuPG provides several types of signatures, which basically differ in the format these have. First we want you to provide a *detached signature* file over the confidential message that you just created userid.confidential.gpg.

Make sure that your signature file is named userid.confidential.gpg.sig and that you've use your the secret key you generated to create the signature. To do so, just *verify* the signature with gpg. You should see your email address in good signature.

gpg --verify userid.confidential.gpg.sig

gpg: assuming signed data in 'userid.confidential.gpg'

gpg: Signature made Thu Jun 13 17:26:01 2024 EDT

gpg: using EDDSA key A68CB0AAC8C2CD6D1EF049741D9E018225193CC3

gpg: Good signature from "CY5001 Class Key <cy5001@none.com>" [ultimate]

Finally, *Gnupg* also provides "signed and encrypted" option at once. Read the GPG documentation to figure out how to do this. Use man gpg and gpg --help as well as internet resources to figure out how to do this. To complete this task you need to create confidential and authenticated ciphertext userid.confidentialandauthentic.gpg, which is signed with your secret key and encrypted with the public key of the class. When we decrypt your message with the CY5001 class secret key we need to see something like this.

gpg -d userid.confidentialandauthentic.gpg

gpg: encrypted with 256-bit ECDH key, ID EC0493472C23A5AF, created 2024-06-13

"CY5001 Class Key <cy5001@none.com>"

Marisa Sonrisa

pub ed25519/1D9E018225193CC3

CY5001 <semester> <year>pub

gpg: Signature made Thu Jun 13 18:59:26 2024 EDT

gpg: using RSA key 2F486D91ACC01232BFEBB6EE83D464CB1ED7A11D7

gpg: Good signature from "Student <userid@northeastern.edu>" [unknown]

gpg: WARNING: This key is not certified with a trusted signature!

gpg: There is no indication that the signature belongs to the owner.

Primary key fingerprint: 8F48 6D91 ACC0 350B FEBB 6EE8 3D46 4CB1 ED7A 11D7

The assignment clearly states that messages needs to be **signed and encrypted at once**. That does not mean sign first and then encrypt it, or encrypt first and then sign it.

Task 3: Certification of Public keys

The first part of this lab introduced you to the classical way of setting up your public key using a file or a keyserver. There are several public databases of gpg/pgp public keyservers available, so you can directly export/import keys from there.

We will be using the Ubuntu OpenPGP Public KeyServerLinks to an external site.. maintained at https://keyserver.ubuntu.com/, although there are many others.

You can import CY5001 Class Key (cy5001@none.com) from here:

https://keyserver.ubuntu.com/pks/lookup?op=vindex&search=0x1D9E018225193CC3
Once you have it you can verify its fingerprint and, if it is correct, sign it with your private key.

When you certify a public key, you sign it with your secret key, so you endorse that the public key is valid (actually belongs to the identity associated with it). Signing the class public key you are affirming that the public key you have it is actually the public key of the class. To do so you need to verify that the fingerprint of your downloaded/copy&paste key is the same like the one stored in the Ubuntu Keyserver. The following command shows fingerprint and KeyID of a gpg key.

```
$ gpg --fingerprint --keyid-format=short 25193CC3

pub   ed25519/25193CC3 2024-06-13 [SC] [expires: 2026-06-13]

      Key fingerprint = A68C B0AA C8C2 CD6D 1EF0  4974 1D9E 0182 2519 3CC3

uid                 [ultimate] CY5001 Class Key <cy5001@none.com>

sub   cv25519/EC0493472C23A5AF 2024-06-13 [E]
```

Once you verify that the public key is correct, you can proceed to certify it (you will be prompted to verify *fingerprint*again). Once it is done, Class public key should have added a new signature.

```
gpg --list-signatures 25193CC3

pub   ed25519 2024-06-13 [SC] [expires: 2026-06-13]

      A68CB0AAC8C2CD6D1EF049741D9E018225193CC3

uid           [ultimate] CY5001 Class Key <cy5001@none.com>
```

sig 3 1D9E018225193CC3 2024-06-13 CY5001 Class Key <cy5001@none.com>

sig B258CB1E8A3A21CA 2024-06-13 Student <userid@northeastern.edu>

sub cv25519 2024-06-13 [E]

sig 1D9E018225193CC3 2024-06-13 CY5001 Class Key <cy5001@none.com>

Now, once you have signed the class public key, send the signed copy of the Class Public Key back to the keyserver, use:

gpg --keyserver keyserver.ubuntu.com --send-keys ....

Remember that you need to properly complete the previous command. Also make sure that your signature is being exported and it appears in the listed signatures of the class key present on the keyserver.

Task 4: Decrypt a secure message sent to you

In this part of the assignment you will be decrypting a ciphered message that is stored in the shared folder /home/sharedfolder/proj2.1 of the master server. (cy5001P21.mscy.org Port:10010). You need to access to the master server to transfer (SCP) the stored message file, named <your userid>.enc.

The <your userid>.enc file has been created while being encrypted with a temporary key pair that we generated for you. The files containing your temporary key pair can also be found in the shared folder (<your userid>.puband <your userid>.sec). You will need those keys in order to access to your secret message.

> Your secret key (private key) is password protected and we have used
the same password that we provided you to access the master server over SSH.

Finally, If you are able to decrypt your individual encrypted file <your userid>.enc, you will gain access to a cookie (a random number), a text file containing that cookie is the deliverable of this task of the assignment. Decrypted file need to named <your userid>.plain and it will be your deliverable of this task.

Your secret message was signed as well. In order to verify the correct signature, you will need to import a public key stored in the master server "sharedfolder/proj2.1/sp26lab2.1.pub"

Deliverables

Submit the files listed below to **Gradescope**. Remember that names need to match exactly extensions (lowercase) and your userid is your username in your username@northeastern.edu email address.

- Task 1
    - userid.key.pub
    - userid.formyeyesonly.gpg
- Task 2
    - userid.confidential.gpg
    - userid.confidential.gpg.sig
    - userid.confidentialandauthentic.gpg
- Task 3
    - Upload your signed version of the Class key to the keyserver
- Task 4
    - userid.plain

Grading

This project grading is broken down as follows (out of 100):

Task	Points	Description
TASK1	20 points	Turning in a 4096 bit, RSA public key that includes your name and northeastern.edu email address
		Submitting an encrypted message using your just created public key
TASK2	40 points	Turning a proper confidential message encrypted with the class public key
		Providing a detached signature using your private key
		Submitting a confidential and authentic message
TASK3	10 points	Properly updating your signature of the CY5001 class on the keyserver
TASK4	30 points	Submitting the decrypted cookie file

- 5 Points will be deducted for each files in incorrect naming or format (e.g. not ASCII).

- Having signatures that are not verifiable, encrypting messages using the wrong keys, etc. will be considered as failing that part of the assignment.
- Not being able to submit files into Gradescope and for example send them by email, will represent 30 points deduction.